Evaluation of dependencies in CTFS and fgeo+ctfs

Gabriel Arellano

2026-06-01

Contents

1 Goal and approach

Here we explore dependencies of the (possibly updated) set of functions in the CTFS / fgeo / fgeo2 body of code. One of the problems with the code is the dense network of inter-dependencies. One of the goals of the current evaluation is to reduce dependency to facilitate debugging and improving.

I am using this terminology:

- CTFS: refers to the original distribution of scripts by Rick Condit, not formal, but where (most) of the functionality is. It remains the most stable distribution of functionality. It's in local scripts.
- ctfs: the R package that encapsulates CTFS scripts. Initially created as the "working engine" behind fgeo R package.

- fgeo: the newer R package that has been demonstrated impossible to maintain and debug without a dedicated maintainer. Initially, it only wrapped ctfs ~ CTFS, but later reorganized code, including adding or removing functionality. It is way more solid and consistent than CTFS, but highly boureauchratic and cryptic for the final user.

For the most part, I will be following CTFS structure and code, which is more transparent and closer to the final user. I will use fgeo to check for improvements in functionality, more than anything.

In CTFS, there are relatively large sets of functions that seem to be linked to obsolete functionality, such as pseudo-Bayesian modeling or geometrical code to process digitized paper maps of stems. Some of that was not included in fgeo. Identifying those clusters early can facilitate the work.

Because part of the functionality is going to be deprecated, substituted by new code, or delegated into other packages, it is useful to work from high-level functions (they depend on others, but nothing depends on them) to low-level functions (others depend on them, but they don't do much by themselves). If a high-level function gets deprecated, then the dependencies may also be deprecated. Although the order in which the work is being done is not relevant for the user, it can help the current update and future code improvements.

2 Preliminaries

First, we declare the path to fgeo2 folder, load fgeo and ctfs packages, and source useful auxiliary functions:

```
# This is my local path. You may have to adapt this.
g <- regexpr("fgeo2", getwd())
path <- substr(getwd(), 1, g + attr(g, "match.length"))

# Load ctfs+fgeo packages:
#pak::pak("forestgeo/ctfs")
#pak::pak("forestgeo/fgeo")
library(ctfs, quietly = TRUE)
library(fgeo, quietly = TRUE)
```

```
-- Attaching packages ---------------------------------------- fgeo 1.1.4 --

v fgeo.analyze 1.1.15     v fgeo.tool    1.2.10
v fgeo.plot    1.1.11     v fgeo.x       1.1.4
```

```
-- Conflicts ------------------------------------------------ fgeo_conflicts() --
x fgeo.analyze::abundance() masks ctfs::abundance()
x fgeo.tool::filter()       masks stats::filter()
```

```
# Other useful functionality:
source(paste0(path, "evaluation/0-code-to-source.r"))
library(igraph, quietly = TRUE)
```

```
Warning: package 'igraph' was built under R version 4.4.3
```

```
Attaching package: 'igraph'

The following objects are masked from 'package:ctfs':

    neighbors, print_all

The following objects are masked from 'package:stats':

    decompose, spectrum

The following object is masked from 'package:base':

    union
```

3 Dependencies and priorities from CTFS code

3.1 Load CTFS local code

This chunk of code will source all the scripts, something similar to the old "startCTFS" function, but following a different structure of folders:

```
# Source the R scripts
f1 <- list.files(paste0(path, "01_utilities/R/old/"),
                 full.names = TRUE, pattern = "\\.R$", ignore.case = TRUE)
f2 <- list.files(paste0(path, "02_biomass/R/old/"),
                 full.names = TRUE, pattern = "\\.R$", ignore.case = TRUE)
f3 <- list.files(paste0(path, "03_spatial/R/old/"),
                 full.names = TRUE, pattern = "\\.R$", ignore.case = TRUE)
```

```
f4 <- list.files(paste0(path, "04_abundance_diversity/R/old/"),
                 full.names = TRUE, pattern = "\\.R$", ignore.case = TRUE)
f5 <- list.files(paste0(path, "05_demography_growth/R/old/"),
                 full.names = TRUE, pattern = "\\.R$", ignore.case = TRUE)
f6 <- list.files(paste0(path, "06_topography_habitat/R/old/"),
                 full.names = TRUE, pattern = "\\.R$", ignore.case = TRUE)
f7 <- list.files(paste0(path, "07_mapping_plotting/R/old/"),
                 full.names = TRUE, pattern = "\\.R$", ignore.case = TRUE)

ff <- c(f1, f2, f3, f4, f5, f6, f7)
for(f in ff) source(f)
```

3.2 Map CTFS dependencies

This maps all the mutual dependencies among the CTFS body of functions (in the global environment):

```
# All functions in the target body of code:
all <- unique(unlist(lapply(ff, function(f) functions_in_script(f))))

# All of the dependencies among all functions in the global environment:
edges <- get_all_dependencies()

# Subset the dependencies for the target functions only:
keep <- edges$from_function %in% all | edges$to_function %in% all
edges <- edges[keep,,drop = FALSE]
edges <- edges[, c("from_function", "to_function")]
colnames(edges) <- c("from", "to")

# Proper graph representation, maybe useful:
g <- graph_from_data_frame(edges, directed = TRUE)
g.ctfs <- g # we make a copy for comparison later

pdf(paste0(path, "evaluation/results/original-CTFS-dependencies.pdf"),
    height = 33, width = 33)

plot(g, layout = layout.fruchterman.reingold,
     vertex.label = V(g)$name,
     vertex.size = 5,
     vertex.color= V(g)$color,
     vertex.frame.color= "white",
```

```
     vertex.label.color = "black",
     edge.width=E(g)$weight,
     edge.color="black")

dev.off()
```

```
pdf
  2
```

3.3 From high-level to low-level

It would be easier to review the code from high to low-level functions:

- High-level functions: nothing depends on them, they depend on others
- Low-level functions: they do not depend on others, they do not do interesting stuff

The reason is to alleviate the body of low-level functions if some of the high-level functions are deprecated or substituted by others with fewer dependencies.

```
# Count number of dependencies and dependents of each function:
n_up <- sapply(all, function(x) length(upstream_dependencies(x, edges)))
n_down <- sapply(all, function(x) length(downstream_dependents(x, edges)))

# Rough rank from high to low level:
how_high <- (n_up / max(n_up)) - (n_down / max(n_down))
as.matrix(head(sort(how_high, decreasing = TRUE), 20))
```

```
                              [,1]
fitSeveralAbundModel     1.0000000
run.growthbin.manyspp    0.9523810
model.littleR.Gibbs      0.8803717
run.growthfit.bin        0.8327526
lmerBayes                0.7619048
growth.flexbin           0.7131243
binGraphManySpecies.Panel 0.6190476
binGraphManySpecies      0.5470383
compare.growthbinmodel   0.4274100
modelBayes               0.3809524
abund.manycensus         0.3809524
graph.abundmodel         0.3809524
graph.outliers           0.3809524
```

```
pdf.allplot               0.3809524
png.allplot               0.3809524
map2species               0.3809524
full.abundmodel.llike     0.3797909
pt.to.segment             0.3333333
biomass.CTFSdb            0.3333333
fullplot.imageJ           0.3333333
```

3.4 Isolated sets of functions

Although most of the code belongs to one network of interdependent code, there are several sets of functions that are disconnected from the rest. Early on, these are potentially low-hanging fruits for review and development. Later, we expect more self-contained modules of intermediate size.

```
# Clusters of functions:
cl <- components(g, mode = "weak")
table(table(cl$membership))
```

```
  2   3   4   7 197
 15   6   1   1   1
```

```
lapply(seq_len(cl$no)[-which.max(cl$csize)], function(i) {
  as.matrix(attr(cl$membership, "names")[cl$membership == i])
})
```

```
[[1]]
     [,1]
[1,] "CalcRingArea"
[2,] "partialcirclearea"
[3,] "circlearea"
[4,] "Annuli"
[5,] "NDcount"
[6,] "NeighborDensities"
[7,] "RipUvK"

[[2]]
     [,1]
[1,] "ba"
[2,] "basum"
```

```
[[3]]
     [,1]
[1,] "beta.total"
[2,] "beta.normalized"
[3,] "fit.beta.normal"

[[4]]
     [,1]
[1,] "dbeta.reparam"
[2,] "betaproduct"

[[5]]
     [,1]
[1,] "bootconf"
[2,] "bootstrap.corr"

[[6]]
     [,1]
[1,] "calc.directionslope"
[2,] "calc.gradient"

[[7]]
     [,1]
[1,] "calcS.alpha"
[2,] "d.calcS.alpha"
[3,] "calcalpha"

[[8]]
     [,1]
[1,] "match.dataframe"
[2,] "CountByGroup"
[3,] "rearrangeSurveyData"

[[9]]
     [,1]
[1,] "order.by.rowcol"
[2,] "order.bynumber"
[3,] "DBHtransition"

[[10]]
     [,1]
[1,] "exponential.sin"
```

```
[2,] "dexp.sin"
[3,] "pexp.sin"

[[11]]
     [,1]
[1,] "dgammaMinusdexp"
[2,] "dgammaPlusdexp"
[3,] "rsymexp"
[4,] "dgammadexp"

[[12]]
     [,1]
[1,] "dpois.max"
[2,] "dpois.maxtrunc"

[[13]]
     [,1]
[1,] "segmentPt"
[2,] "drawrectangle"

[[14]]
     [,1]
[1,] "circle"
[2,] "fullcircle"

[[15]]
     [,1]
[1,] "ellipse"
[2,] "fullellipse"

[[16]]
     [,1]
[1,] "full.xygrid"
[2,] "graph.mvnorm"

[[17]]
     [,1]
[1,] "draw.axes"
[2,] "imageGraph"

[[18]]
     [,1]
[1,] "arrangeParam.llike.2D"
```

```
[2,] "lmerBayes.hyperllike.sigma"

[[19]]
     [,1]
[1,] "dmixnorm"
[2,] "minum.mixnorm"

[[20]]
     [,1]
[1,] "qasympower"
[2,] "rasympower"

[[21]]
     [,1]
[1,] "Gibbs.regsigma"
[2,] "Gibbs.regslope"
[3,] "regression.Bayes"

[[22]]
     [,1]
[1,] "se.skewness"
[2,] "se.kurtosis"

[[23]]
     [,1]
[1,] "getTopoLinks"
[2,] "solve.topo"
```

4 Dependencies in fgeo + ctfs packages

As a potentially useful reference, I also map the dependencies involving `fgeo` + `ctfs` R packages. Remember that `fgeo` is a wrapper that loads four packages:

- `fgeo.tool`
- `fgeo.x`
- `fgeo.analyze`
- `fgeo.plot`

```
# Dependencies of fgeo + ctfs packages:
edges <- get_all_dependencies(
  packages = c("fgeo", "fgeo.tool", "fgeo.x", "fgeo.analyze", "fgeo.plot", "ctfs"),
```

```
  include_global = FALSE,
  internal = TRUE
)

edges <- edges[, c("from_id", "to_id")]
colnames(edges) <- c("from", "to")

# Proper graph representation:
g <- graph_from_data_frame(edges, directed = TRUE)

# Visual representation:
V(g)$color <- rep("gray", length(V(g)))
V(g)$color[grepl("fgeo.tool::", V(g)$name)] <- "gold"
V(g)$color[grepl("fgeo.x::", V(g)$name)] <- "gold"
V(g)$color[grepl("fgeo.analyze::", V(g)$name)] <- "gold"
V(g)$color[grepl("fgeo.plot::", V(g)$name)] <- "gold"
V(g)$color[grepl("ctfs::", V(g)$name)] <- "orange"

pdf(paste0(path, "evaluation/results/fgeo+ctfs-dependencies.pdf"),
    height = 33, width = 33)

plot(g, layout = layout.fruchterman.reingold,
     vertex.label = V(g)$name,
     vertex.size = 5,
     vertex.color= V(g)$color,
     vertex.frame.color= "white",
     vertex.label.color = "black",
     edge.width=E(g)$weight,
     edge.color="black")

dev.off()
```

```
pdf
  2
```

The newer version of the code increases the number of functions and number of dependencies between functions (in addition to dependencies to other packages, not considered here):

- Number of functions: 256 in CTFS, *vs.* 581 in `fgeo` + `ctfs`.
- Number of dependencies: 318 in CTFS, *vs.* 714 in `fgeo` + `ctfs`.